IW AI Core Platform — Architecture Diagram

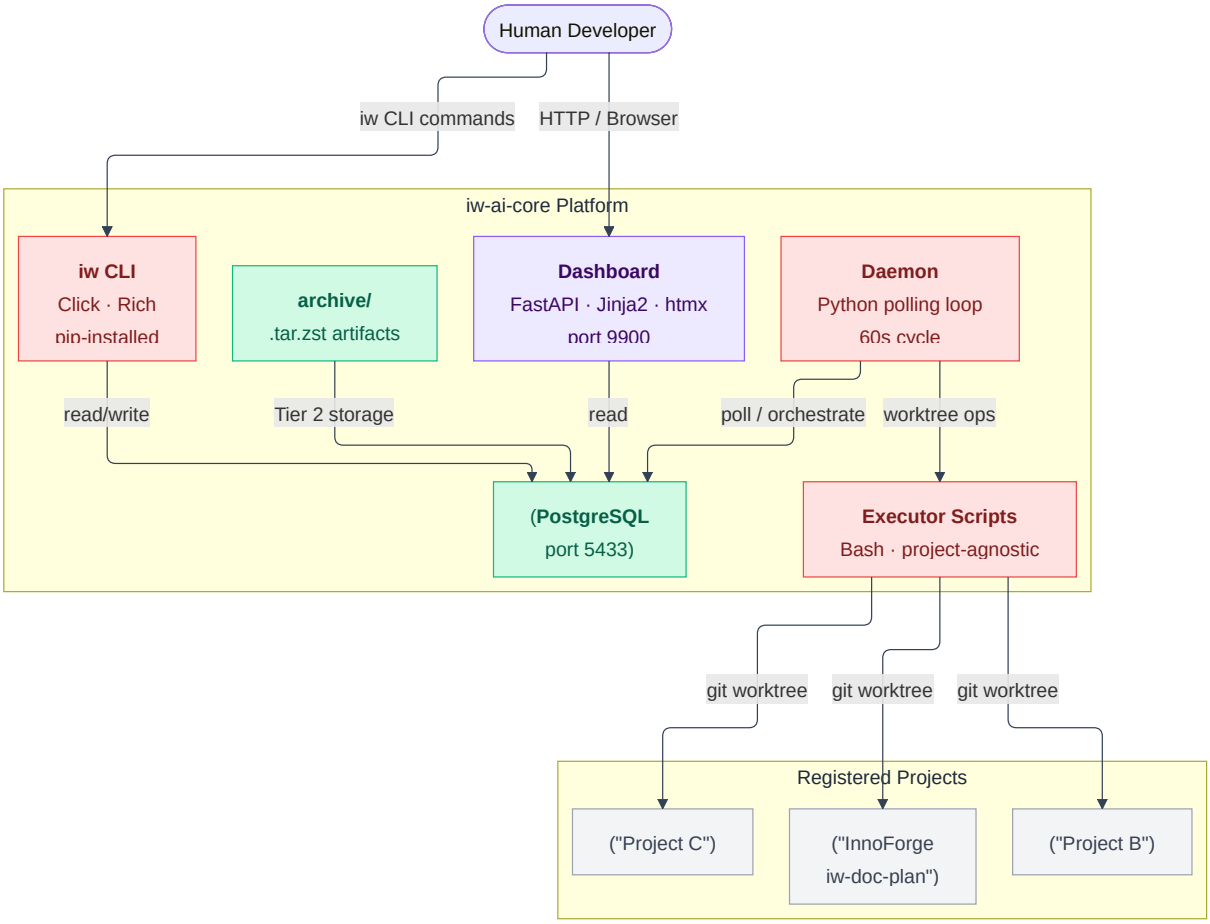
Type: Diagram **Version:** v12 **Status:** Published **Generated:** 2026-05-11 21:33:46

IW AI Core Platform — Architecture Diagram

Project: IW AI Core
Document ID: diagram-architecture
Editorial Category: technical
Generated: 2026-05-11

System Architecture Overview

> ***Why this diagram?*** *Use this to understand which physical components exist, how the iw-ai-core platform relates to managed projects, and where each type of traffic (human UI, agent CLI, daemon orchestration) flows.*

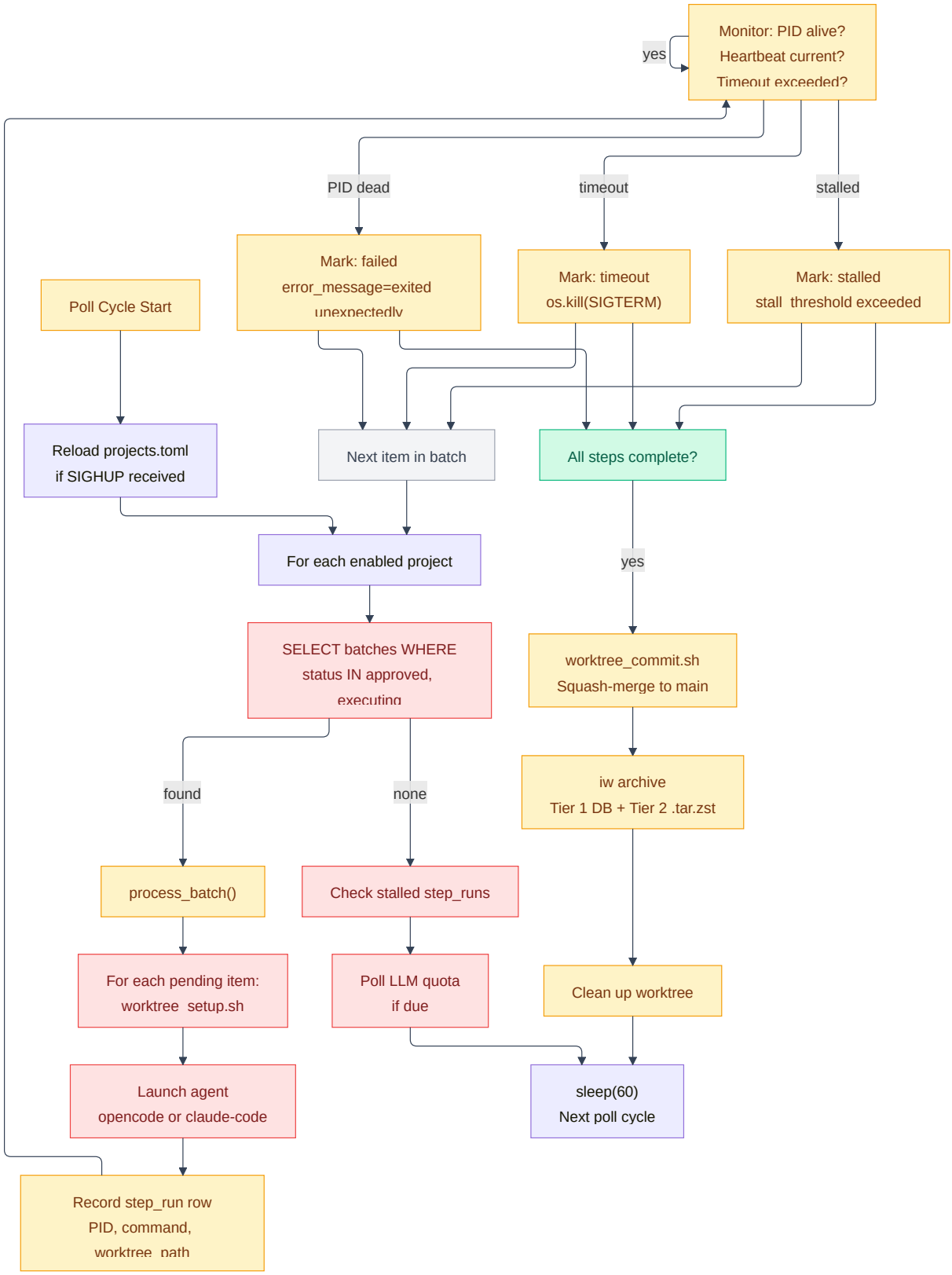


Key topology facts:
- The platform runs **one daemon process** managing **all registered projects** from a single PostgreSQL instance on port 5433.
- Each project repo is a separate `git worktree` root. The daemon never runs inside a managed project repo —

it operates on worktrees it creates.
- The `iw CLI` is installed once on the developer machine and bridges agents to the database. Agents call `iw step-done`, `iw next-id`, and other commands from inside their worktrees.

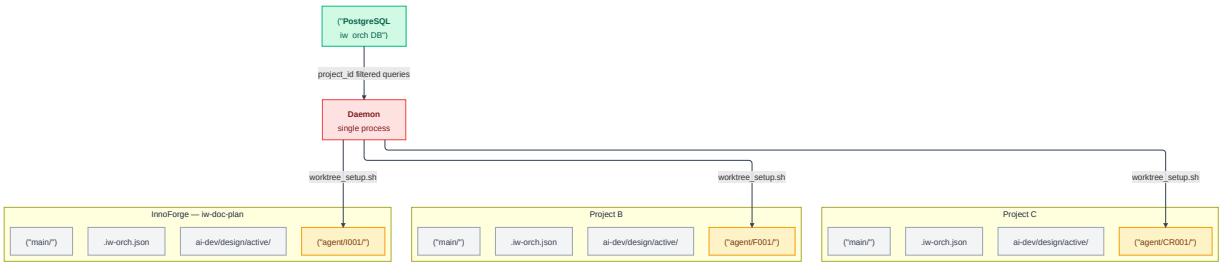
Daemon Execution Pipeline

> ***Why this diagram?*** *Use this to understand the daemon's poll loop decision tree: when it picks up a batch, how it monitors running agents, when it triggers a timeout or stall, and how it proceeds to merge and archive.*



Multi-Project Isolation

> ***Why this diagram?*** *Use this to see how a single PostgreSQL instance, daemon, and dashboard serve multiple isolated projects simultaneously, with worktrees and design files kept strictly within each project's own repository.*

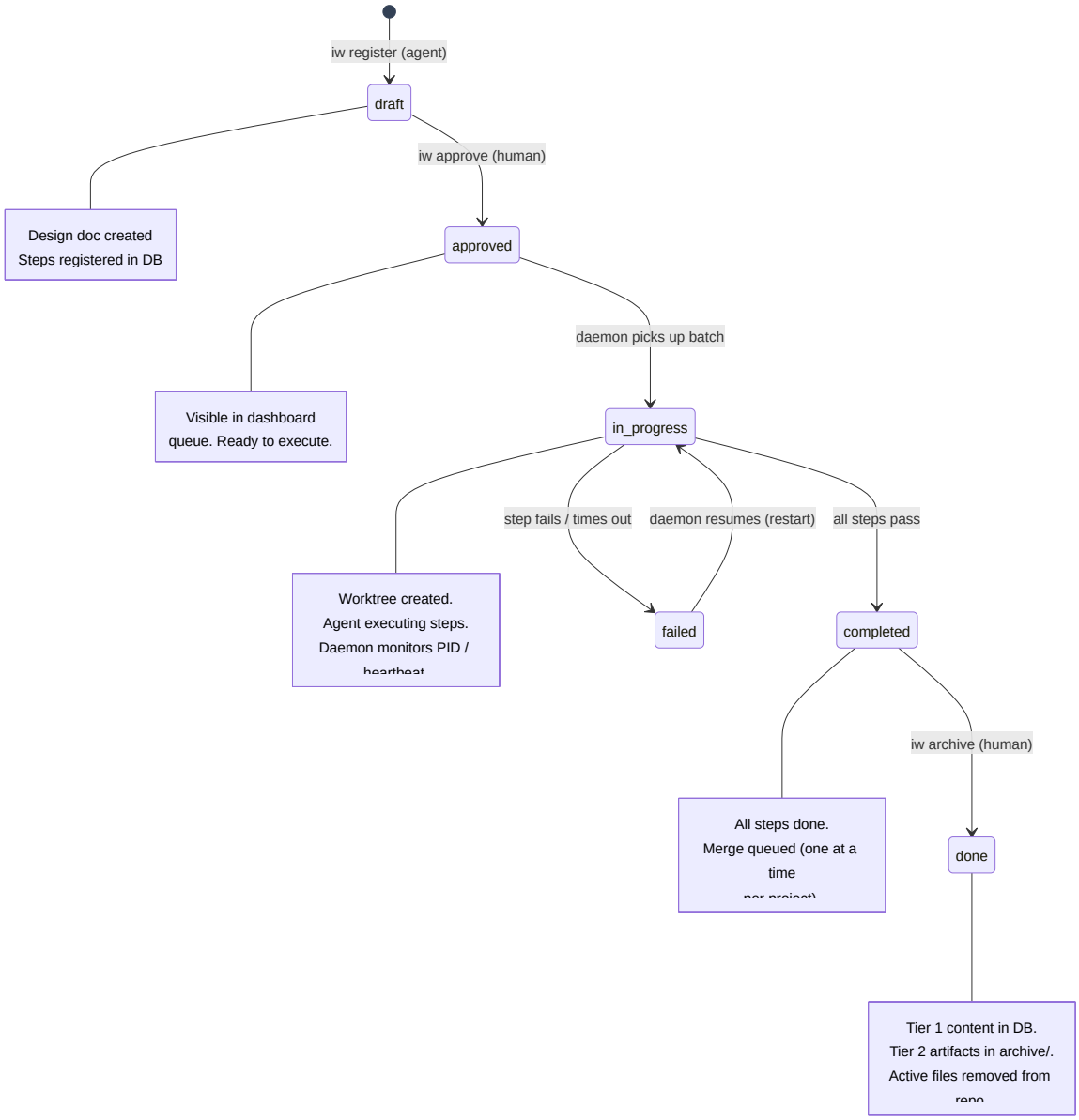


Isolation guarantees:

- Every table uses `project_id` as a filter. InnoForge's `I001` never collides with Project B's `I001`.
- Worktrees live inside each project's own repo. The daemon never creates a worktree in the platform repo.
- Merge queues are per-project. An InnoForge merge never blocks a Project B merge.
- LLM quota is shared globally — all projects compete for the same Claude API budget, tracked by the quota monitor.

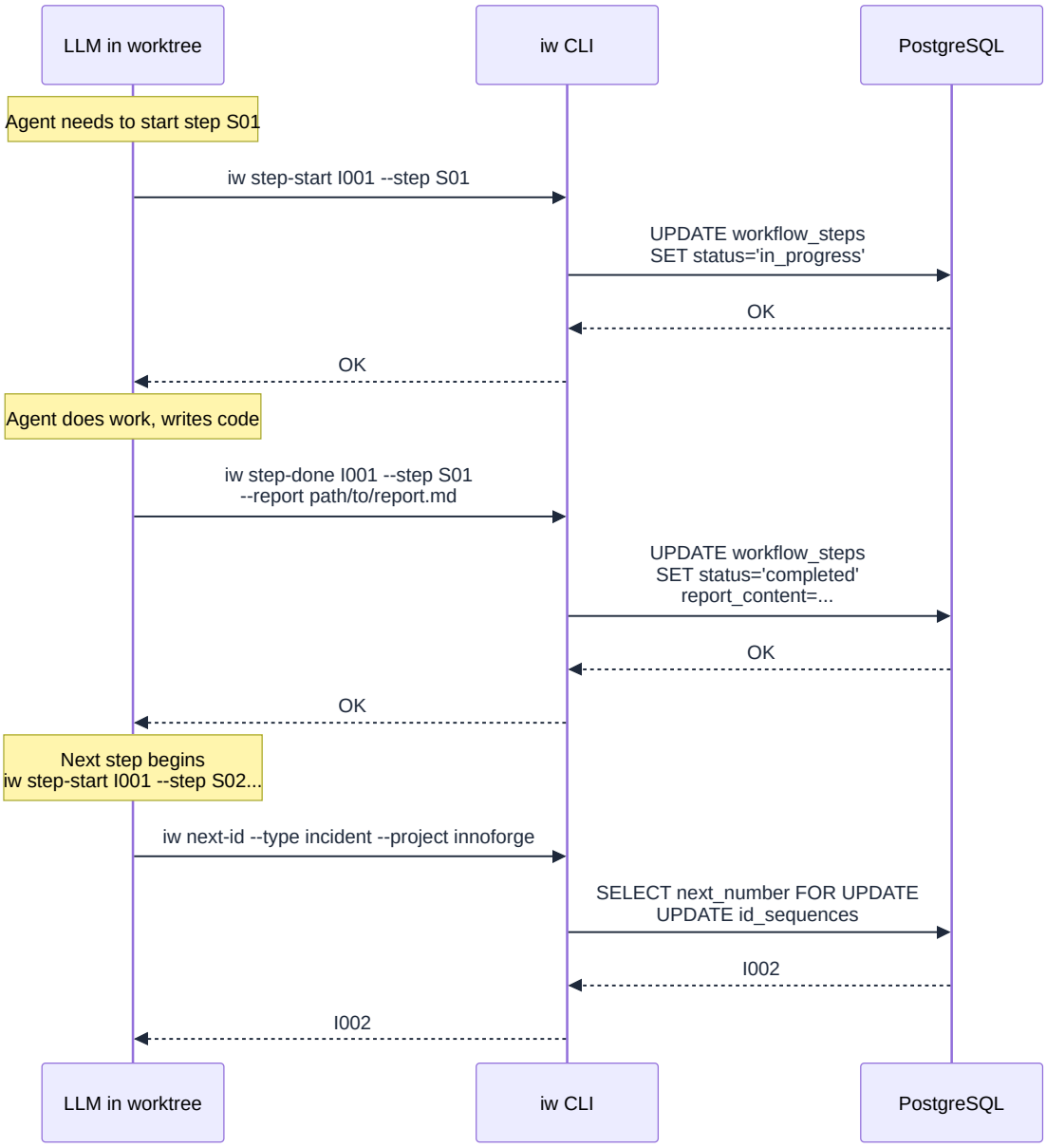
Work Item State Machine

> ***Why this diagram?*** *Use this to trace a work item from creation through execution to completion, and understand which actors (human, agent, daemon) drive each transition.*



Agent-to-Database Communication Pattern

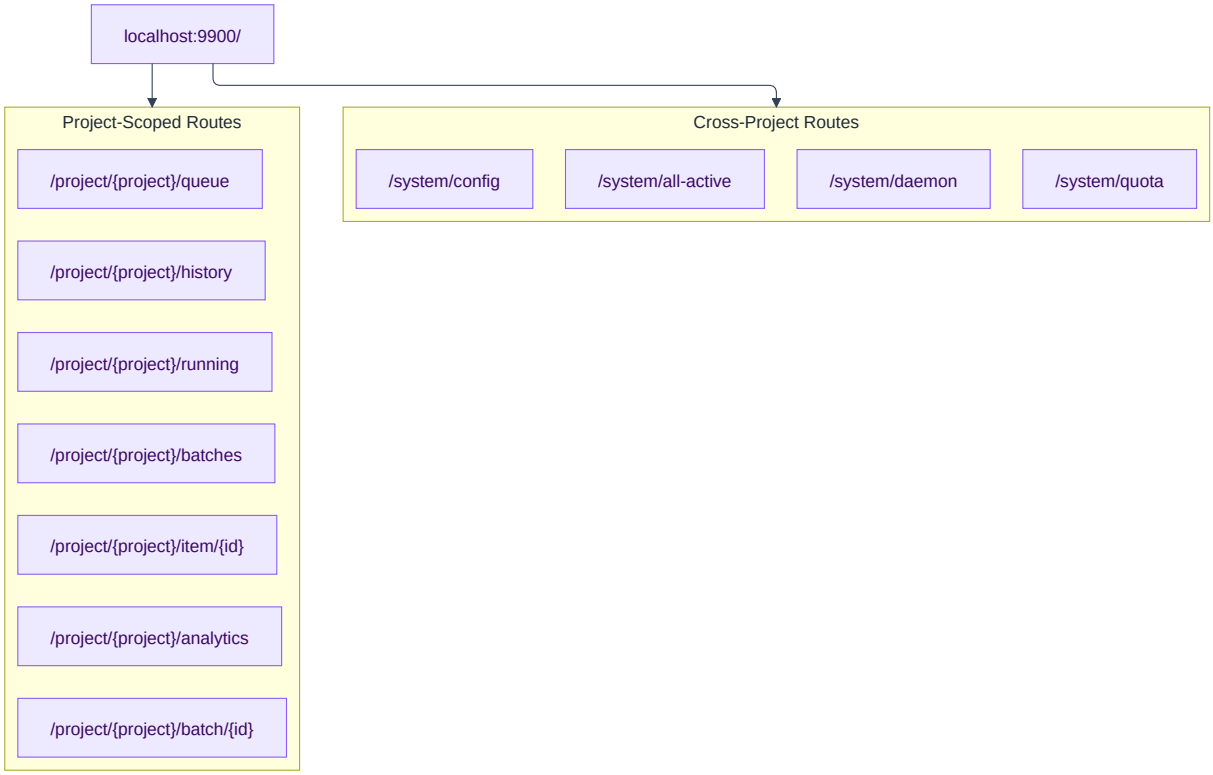
> ***Why this diagram?*** *Use this when tracing a step execution and understanding how agents (LLMs in worktrees) communicate with the database exclusively through the* `iw` *CLI — never directly.*



Key architectural principle: Agents are LLM sessions without database credentials. They communicate through the `iw` CLI which provides a versioned, validated interface. The DB is never exposed directly to an agent process.

Dashboard Information Architecture

> ***Why this diagram?*** *Use this to understand how the FastAPI dashboard routes are organized per project, with cross-project views available at* `/system/` *.*



All project-scoped routes filter their DB queries by `project_id`. The sidebar project selector changes the URL prefix. SSE streams at `/sse` push real-time updates to all connected dashboard clients.
